Implementacja algorytmu mnożenia liczb dowolnej precyzji

Cybulski Mateusz, Derron Tomasz, Domiński Bartosz

Streszczenie

Implementacja algorytmu mnożenia Toom-Cook'a (Toom-3) dla liczb całkowitych dowolnej precyzji w C++. Zaprojektowanie i przeprowadzenie eksperymentu mającego na celu porównanie utworzonej implementacji z istniejącymi bibliotekami: GMP i wbudowaną biblioteką w Python.

1 Metodologia

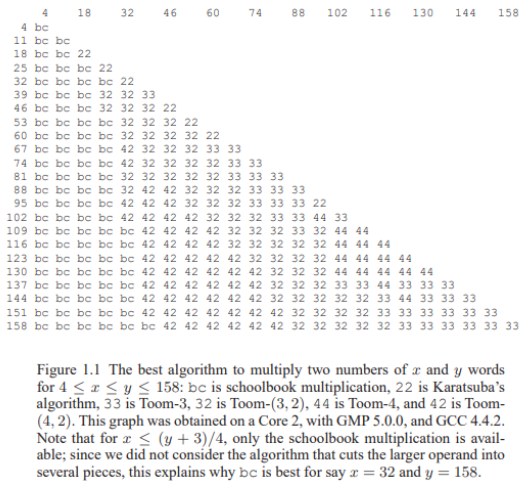
1.1 Cel eksperymentu

Celem eksperymentu jest zbadanie zależności pomiędzy czasem wykonania operacji mnożenia a rozmiarem danych wejściowych dla utworzonej implementacji algorytmu mnożenia dużych liczb. W drugiej części eksperymentu, na podstawie otrzymanej zależności, nastąpi porównanie naszej implementacji z istniejącymi bibliotekami.

1.2 Opis eksperymentu

Zbiór liczb użyty w eksperymencie będzie składał się z liczb w zapisie o podstawie $\beta = 2^{32}$. W eksperymencie zostaną użyte liczby z zakresu od liczb 50-słowowych do 1000-słowowych (zakładamy, że słowo = 32 bity).

Będziemy mierzyć czas wykonania operacji mnożenia dla liczb o długości 50, 100, 150, ... , 950, 1000 słów. Dla każdej z podanych długości zostanie wygenerowanych 100 liczb o zadanej długości, następnie w pseudolosowy sposób zostaną dobrane w pary, i na tych parach (50 par dla każdej z ustalonych długości) zostanie przeprowadzone mnożenie. Zakres długości liczb został wybrany na podstawie wykresu [**?**], pokazanego poniżej.



	4	18	32	46	60	74	88	102	116	130	144	158
4	bc											
11	bc	bc										
18	bc	bc	22									
25	bc	bc	bc	22								
32	bc	bc	bc	bc	22							
39	bc	bc	bc	32	32	33						
46	bc	bc	bc	32	32	32	22					
53	bc	bc	bc	bc	32	32	32	22				
60	bc	bc	bc	bc	32	32	32	32	22			
67	bc	bc	bc	bc	42	32	32	32	33	33		
74	bc	bc	bc	bc	42	32	32	32	32	33	33	
81	bc	bc	bc	bc	32	32	32	32	32	33	33	33
88	bc	bc	bc	bc	32	42	42	32	32	32	33	33	33
95	bc	bc	bc	bc	42	42	42	32	32	32	33	33	33	22
102	bc	bc	bc	bc	42	42	42	42	32	32	32	33	33	44	33
109	bc	bc	bc	bc	bc	42	42	42	42	32	32	32	33	32	44	44
116	bc	bc	bc	bc	bc	bc	42	42	42	42	32	32	32	32	44	44	44
123	bc	bc	bc	bc	bc	bc	42	42	42	42	42	32	32	32	32	44	44	44	44
130	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	44	44	44	44	44
137	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	33	33	44	33	33	33
144	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	32	32	33	44	33	33	33
151	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	42	32	32	32	32	33	33	33	33	33	33
158	bc	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	32	32	33	33	33	33	33	33

Figure 1.1 The best algorithm to multiply two numbers of x and y words for $4 \leq x \leq y \leq 158$: bc is schoolbook multiplication, 22 is Karatsuba's algorithm, 33 is Toom-3, 32 is Toom-$(3, 2)$, 44 is Toom-4, and 42 is Toom-$(4, 2)$. This graph was obtained on a Core 2, with GMP 5.0.0, and GCC 4.4.2. Note that for $x \leq (y + 3)/4$, only the schoolbook multiplication is available; since we did not consider the algorithm that cuts the larger operand into several pieces, this explains why bc is best for say $x = 32$ and $y = 158$.

Rysunek 1: Algorytm mnożenia stosowany w zależności od długości słów.

Do wygenerowania pseudolosowych liczb jako próbek o zadanej długości zostanie użyty moduł *random*.

Następnie zostanie zbadana zależoność czasu potrzebnego do przeprowadzenia operacji mnożenia w zależności od tego jak duże są mnożone liczby dla utworzonej przez nas implementacji.

Następnie te same pary liczb zostaną pomnożone z wykorzystaniem biblioteki GMP (GNU Multiple Precision Arithmetic Library) oraz w języku Python. Umożliwi to wyznaczenie analogicznej zależności jak dla naszej implementacji, oraz ich porównanie.

1.3 Plan osiągnięcia celów projektowych

Szkielet implementacji

- Fork szkieletu implementacji – znalezienie dostępnego rozwiązania i wykorzystanie go zgodnie z licencją.
- Edycja kodu – usunięcie zbędnego kodu, zmodyfikowanie pod własne potrzeby.

Implementacja dodawania BigInt

- Algorytm dodawania – niezbędny do poprawnego działania mnożenia.
- Testy poprawności – weryfikacja wyników.

Algorytm mnożenia

- Implementacja Toom-3 – wykorzystując wcześniejszy szkielet.
- Testy poprawności – porównanie wyników z bibliotekami.

Eksperyment i benchmarki

- Przygotowanie próbek – z użyciem gotowej biblioteki do generowania liczb pseudolosowych i zapis danych wejściowych według ustalonej metodologii.
- Testy własnej implementacji – pomiar czasu mnożenia próbek.
- Testy rozwiązań GMP – benchmark względem profesjonalnego standardu C++.
- Testy rozwiązań wbudowanych w Pythonie – pomiar czasu w Pythonie.
- Zapisanie wyników – względem ustalonej metodologii (.csv).

Analiza wyników

- Wykresy i analiza danych – wykorzystanie bibliotek Python.
- Wnioski końcowe – podsumowanie efektywności algorytmu w praktyce.

Literatura